

Sorting Algorithm Benchmark Report

Insertion Sort vs Selection Sort vs Merge Sort

This report compares measured running times for three sorting algorithms across multiple input patterns and input sizes. The graphs show how runtime changes as n grows, and the tables estimate where merge sort becomes consistently faster.

Repeats per test

10000

Input patterns

7

Algorithms

3

Input range

0-200

Report outputs

Individual PNG graphs saved in results/plots
PDF report saved as results/sorting_graph_report.pdf
Crossover summary saved as results/crossover_summary.csv

Executive summary

Stable crossover points by input type

Input type	Insertion vs Merge	Selection vs Merge
Random	n = 108	n = 77
Sorted	No stable crossover	n = 80
Reverse	n = 44	n = 73
Duplicates	n = 115	n = 84
Nearly Sorted	No stable crossover	n = 83
All Equal	No stable crossover	n = 81
Few Unique	n = 120	n = 84

Stable crossover means merge sort was faster for five consecutive input sizes starting from that value of n.

How to read the graphs

Guide to interpreting the benchmark results

X-axis

The x-axis shows the input size n .

Y-axis

The y-axis shows the average running time in milliseconds.

Lower line

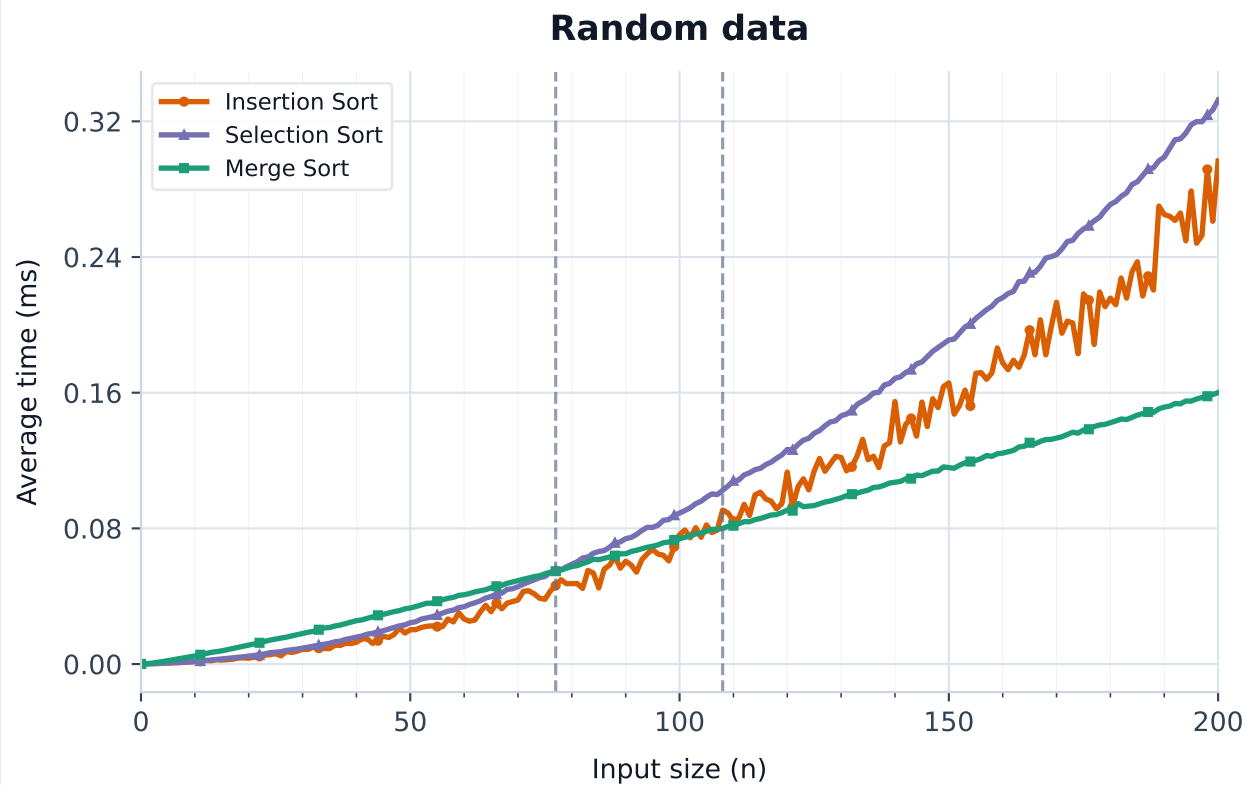
A lower line means the algorithm finished faster.

Dashed line

A dashed vertical line marks a stable crossover point.

Random input

Average running time by input size



Key results

Fastest at largest n

Merge Sort
when $n = 200$

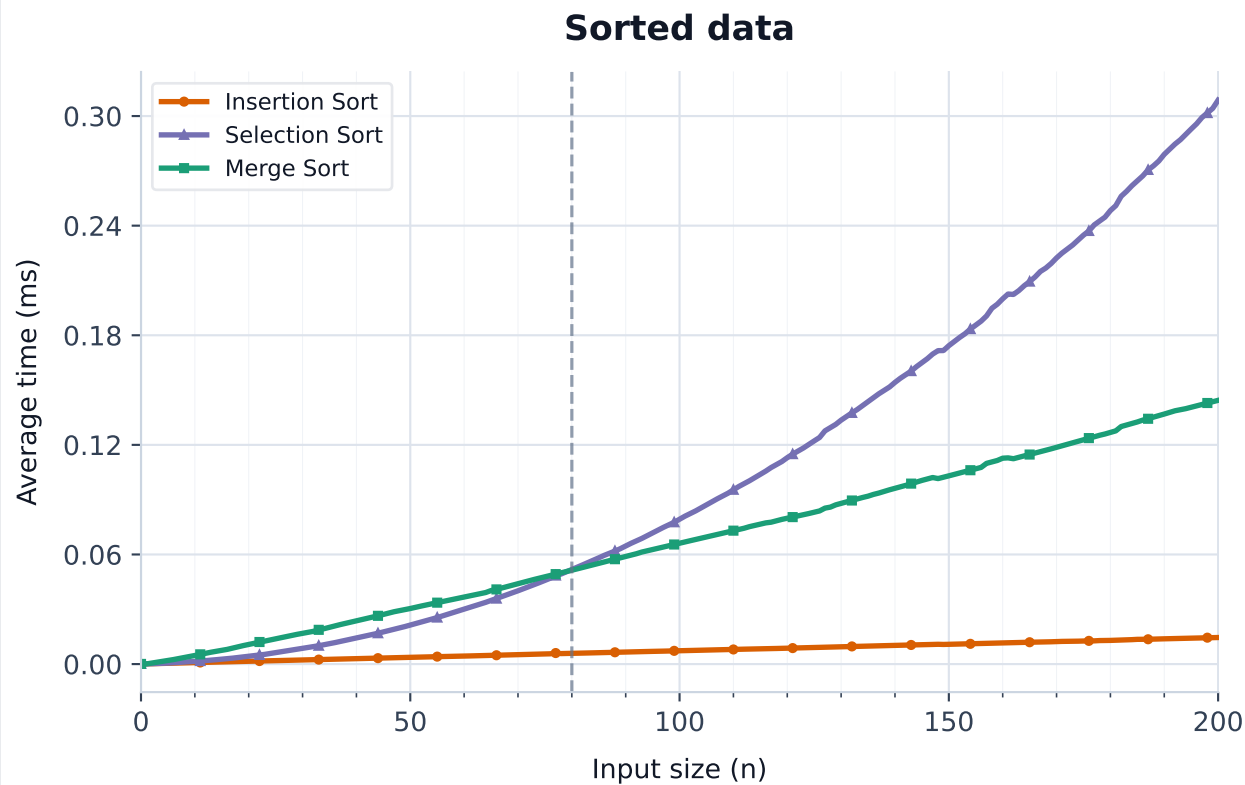
Stable crossovers

Insertion vs Merge
 $n = 108$

Selection vs Merge
 $n = 77$

Sorted input

Average running time by input size



Key results

Fastest at largest n

Insertion Sort
when $n = 200$

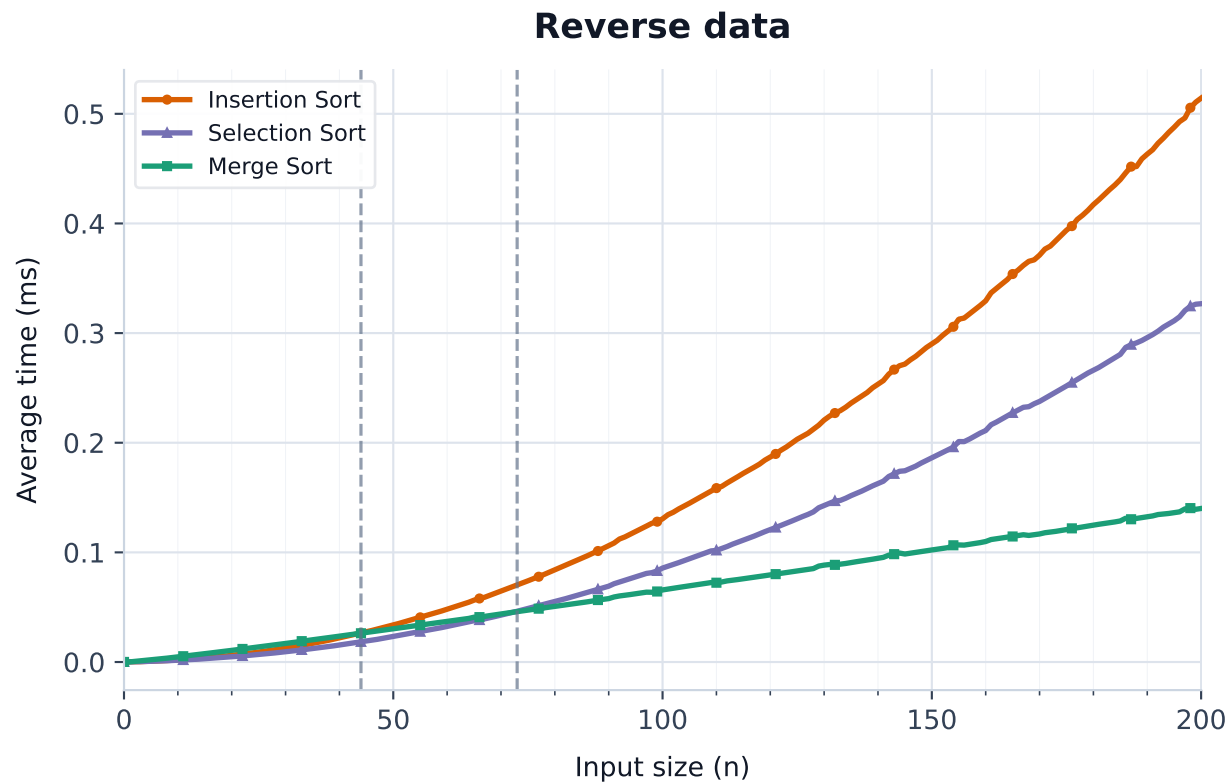
Stable crossovers

Insertion vs Merge
No stable crossover

Selection vs Merge
 $n = 80$

Reverse input

Average running time by input size



Key results

Fastest at largest n

Merge Sort
when $n = 200$

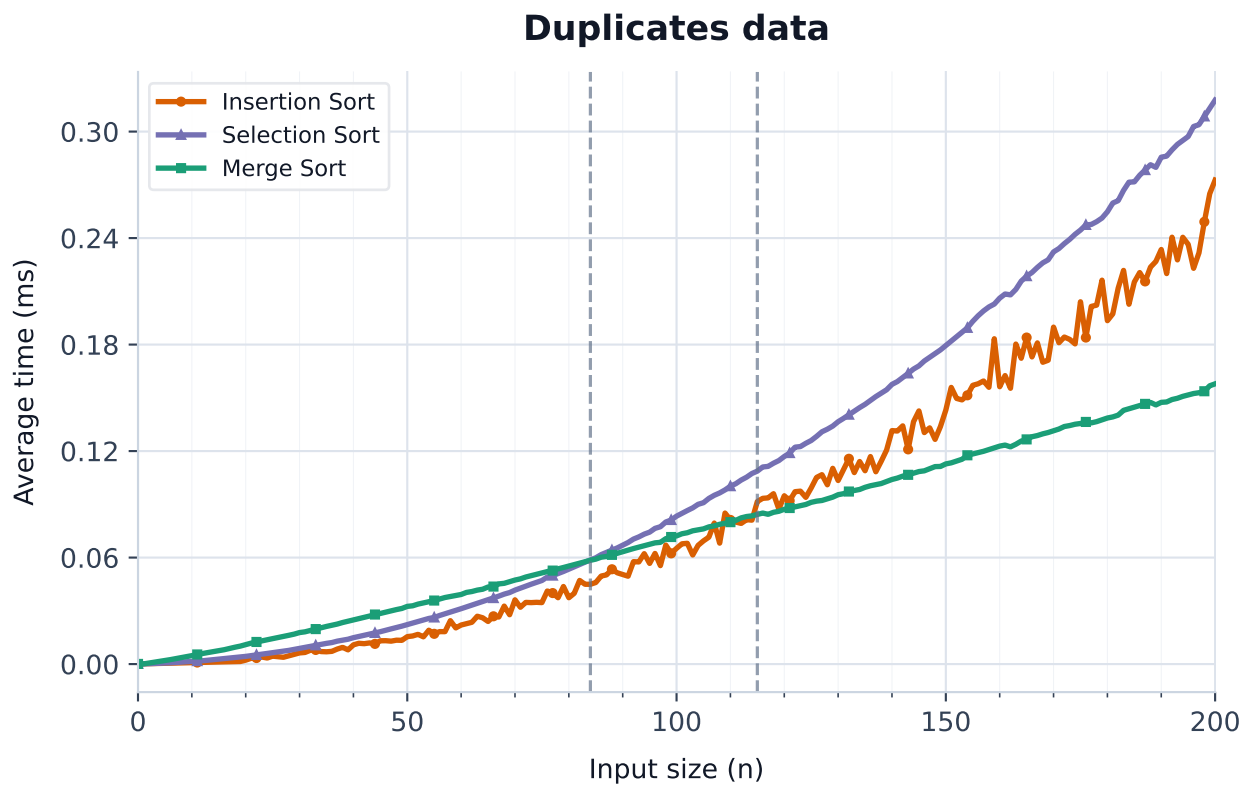
Stable crossovers

Insertion vs Merge
 $n = 44$

Selection vs Merge
 $n = 73$

Duplicates input

Average running time by input size



Key results

Fastest at largest n

Merge Sort
when $n = 200$

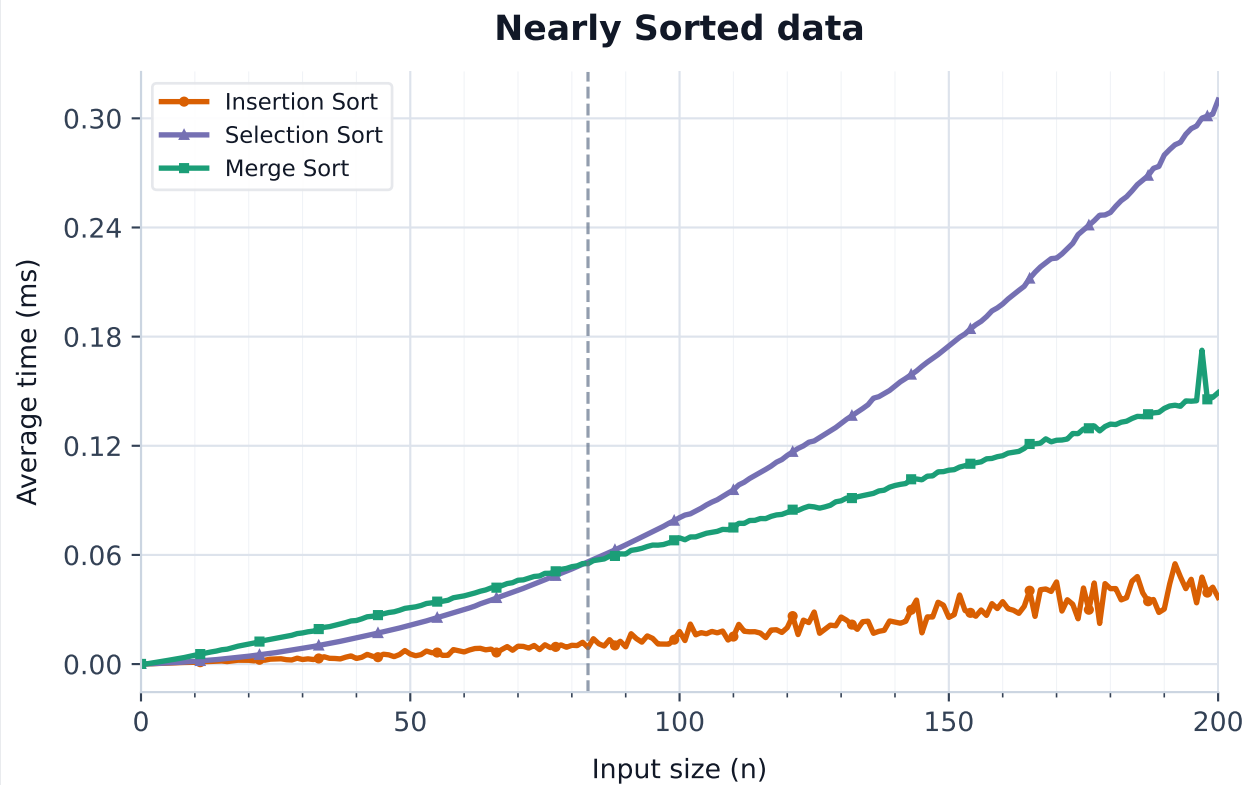
Stable crossovers

Insertion vs Merge
 $n = 115$

Selection vs Merge
 $n = 84$

Nearly Sorted input

Average running time by input size



Key results

Fastest at largest n

Insertion Sort
when $n = 200$

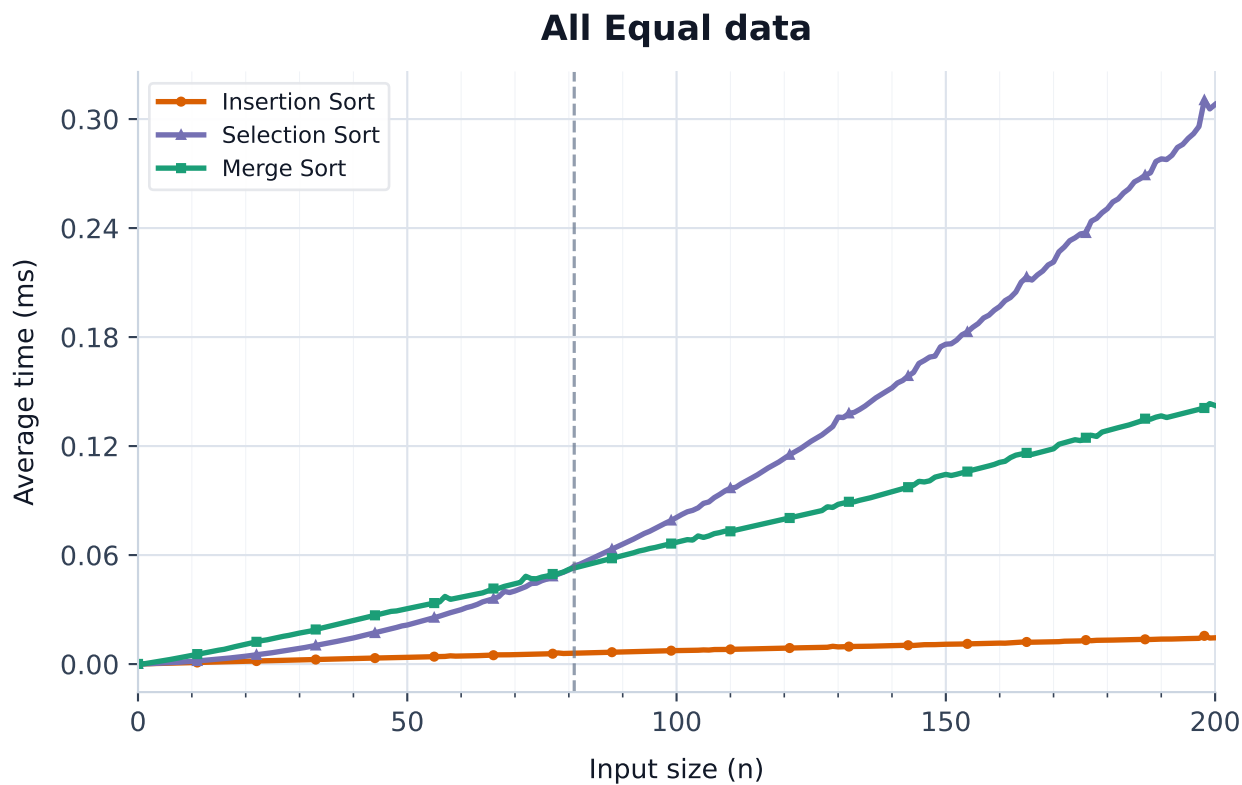
Stable crossovers

Insertion vs Merge
No stable crossover

Selection vs Merge
 $n = 83$

All Equal input

Average running time by input size



Key results

Fastest at largest n

Insertion Sort
when $n = 200$

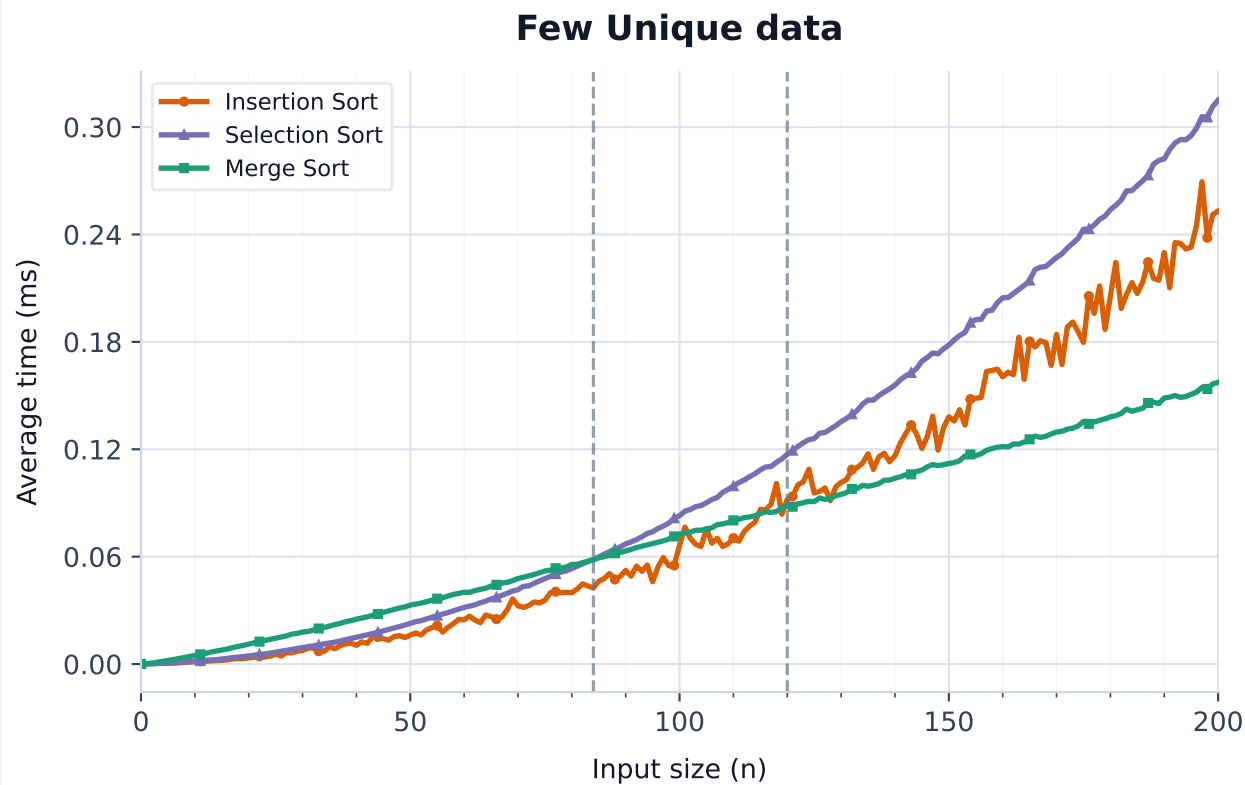
Stable crossovers

Insertion vs Merge
No stable crossover

Selection vs Merge
 $n = 81$

Few Unique input

Average running time by input size



Key results

Fastest at largest n

Merge Sort
when $n = 200$

Stable crossovers

Insertion vs Merge
 $n = 120$

Selection vs Merge
 $n = 84$